Good morning team,

Capstone — Build the ArchintX Coupon Engine

An advanced, full-stack project built in Python — not internship-themed, this one's for ArchintX's own coupon system (the kind behind codes like `MARKETINGINTX26`). Anyone can try a coupon against a price; only ArchintX staff can create or deactivate one. **Frontend + backend + database, deployed and live.**

Today's focus

BUILD

Coupon Engine

Apply-a-coupon page + staff coupon management

GOAL

Rules enforced server-side

Expiry, usage limits and active status all checked in Python, not the browser

> Python Developer team · **multi-day build — submit by 6 PM, Wed 11 Nov 2026**
> Solo build. Starter boilerplate is fine; copying someone else's full project is not.
> No name list here — your **GitHub repo** is what identifies your submission.

1. Frontend

1. **Apply Coupon page** (public, no login) — a coupon code field and a base price field. Submitting shows either the discounted price, or a clear reason it failed (invalid, expired, inactive, usage limit reached).
2. **Staff Login** — email + password, calls your backend's auth API, real error on wrong credentials.
3. **Coupon Dashboard** (staff only) — create a coupon (code, discount %, expiry date, usage limit), and a list of every coupon with how many times it's been used, with a **Deactivate** action on each.
4. A simple server-rendered UI (Flask/Django templates) is completely fine here — this doesn't need to be a JS single-page app. Responsive at desktop and mobile widths either way.
5. All data on screen must come from your API/backend — no hardcoded fake coupons.

◆ 2. Backend

A Python backend — Flask, FastAPI, or Django, your choice. Required endpoints:

1. `POST /auth/login` — staff login, returns an auth token (JWT or session). No public signup — staff accounts are seeded directly in the database.
2. `POST /coupons` — **staff only** — create a coupon: code, discount percent, expiry date, usage limit.
3. `GET /coupons` — **staff only** — list every coupon with its current used-count.
4. `PATCH /coupons/:code/deactivate` — **staff only**.
5. `POST /coupons/:code/apply` — **public** — body: base price. Validate, in this order: code exists → is active → not past its expiry date → under its usage limit. Return the discounted price only if every check passes, otherwise the specific reason it failed.
6. The usage count must only increase on a genuinely successful apply, and must never be allowed past the usage limit even if two requests land at the same moment — handle this with your database's own atomic update/transaction, not a check-then-write in application code.
7. Passwords **hashed** (bcrypt/werkzeug/equivalent) — never stored or logged in plain text. Bad/missing input returns a proper status code (400/401/403/404), not a silent 200.

◆ 3. Database

1. Any database — PostgreSQL, MySQL, MongoDB, SQLite for local dev — your choice, documented with a reason in your README.
2. At least **3 tables/collections**: `coupons` (code, discount percent, expiry date, usage limit, used count, active), `redemptions` (which coupon, base price, discounted price, when), `staff` (email, password hash).
3. Seed data: at least **5 coupons** covering different states — one active and usable, one expired, one at its usage limit, one deactivated — and **1 staff login** for us to test with.
4. Include your schema (table definitions or an ER diagram) in the write-up — we need to see the shape of the data, not just the code.

4. Deployment

1. Backend on a free Python-friendly host — Render, Railway, or Fly.io.
2. Database hosted, not localhost — Supabase, Neon, Railway Postgres, or MongoDB Atlas.
3. If your frontend is a separate static app, host it on Vercel or Netlify; if it's server-rendered by Flask/Django, it ships with the backend — either is fine.
4. Whatever your setup, the deployed frontend must call your **deployed backend**, not `localhost` — double-check this before you submit.

> **A broken deploy is an incomplete submission.** We will try a real coupon against a real price on your live site — if the limit/expiry rules don't actually hold, the task isn't done yet, regardless of how the code looks in the repo.

What to submit

GitHub repo — code, with a README covering setup, your DB schema, and how you made the usage-limit check safe under concurrent requests

Live URL(s) — the deployed app link(s)

Staff test login — one login we can use to create and deactivate a coupon

Screenshots — a successful apply, a rejected apply (your choice of reason), and the staff dashboard

Rules — please follow exactly

1. Passwords must be hashed — never stored, returned by the API, or logged in plain text.
2. Every validation rule (expiry, usage limit, active status) must be enforced in the backend — a frontend that merely hides an expired coupon doesn't count.
3. The usage limit must hold even under repeated/concurrent applies — this is the one thing a real coupon engine cannot get wrong.
4. This is a multi-day build — pace yourself, don't leave deployment for the last hour.

Submission

Submit your repo link, live URL(s), staff login and screenshots by **6 PM, Wednesday 11 November 2026** in the given **Google Drive submission folder**.

Any doubts, message me directly.

Thank you.

HR — ArchintX